HƯỚNG DẪN KIỂM THỬ API

Omnichannel E-Commerce Microservices

Product • Inventory • Order • Payment

> **Mục tiêu:** Giúp người kiểm thử dựng môi trường, gửi request đúng contract, kiểm tra dữ liệu và trạng thái xuyên bốn microservice, đồng thời xác minh validation, idempotency, race condition và luồng sự kiện RabbitMQ.

Thông tin	Giá trị
Phiên bản tài liệu	1.0
Ngày cập nhật	19/08/2026
Base URL	http://localhost:8080
Đối tượng sử dụng	Developer, QA, người bảo vệ/demo đồ án
Nguồn đối chiếu	docs/openapi.yaml, controller/DTO và compose.yaml hiện tại

Tài liệu thực hành - sử dụng Postman hoặc cURL/PowerShell

Cách sử dụng tài liệu

- Nếu cần demo nhanh: làm theo mục 3, 4 và 5 để chạy happy path end-to-end.
- Nếu cần kiểm thử đầy đủ: chạy từng endpoint ở mục 6-9, sau đó chạy các ca lỗi và đồng thời ở mục 10.
- Mọi request công khai đều đi qua Nginx Gateway tại http://localhost:8080.
- Giá tiền dùng kiểu số nguyên long. Ví dụ trong tài liệu dùng VND nên 12.990.000 được gửi là 12990000.

> **Quy ước:** Các giá trị {{...}} là biến Postman. Không nhập nguyên dấu ngoặc nhọn nếu đang test thủ công bằng công cụ khác.

1. Phạm vi và kiến trúc cần biết

Thành phần	Vai trò	Cổng/URL
Nginx Gateway	Điểm vào duy nhất của public API; có rate limit	http://localhost:8080
Product Service	Danh mục, giá và trạng thái sản phẩm	Nội bộ: 8081
Inventory Service	onHand, reserved, reservation	Nội bộ: 8082
Order Service	Order aggregate, snapshot sản phẩm, tổng tiền	Nội bộ: 8083
Payment Service	Thanh toán, state transition, outbox	Nội bộ: 8084
PostgreSQL	4 database độc lập	localhost:15432
RabbitMQ UI	Theo dõi exchange/queue/event	http://localhost:15672
Prometheus	Metrics	http://localhost:9090
Grafana	Dashboard	http://localhost:3000

1.1 Luồng trạng thái chính

Inventory reservation: ACTIVE → CONFIRMED hoặc RELEASED hoặc EXPIRED.

Order: PENDING → INVENTORY_RESERVED → PAYMENT_PROCESSING → CONFIRMED; lỗi có thể đưa order sang FAILED.

Payment: PENDING → SUCCEEDED → REFUNDED, hoặc PENDING → FAILED.

> **Tính eventual consistency:** Sau khi confirm/cancel payment, hãy chờ khoảng 1-3 giây rồi GET order và inventory. Outbox mặc định publish mỗi 1 giây, nên dữ liệu không nhất thiết đổi ngay trong cùng response payment.

2. Ứng dụng và công cụ cần sử dụng

Công cụ	Mức độ	Mục đích
Docker Desktop + Docker Compose	Bắt buộc	Chạy PostgreSQL, RabbitMQ, 4 service, Nginx, Prometheus, Grafana
Postman	Khuyến nghị	Gửi request, lưu biến ID, viết assertions, chạy collection
PowerShell 7 hoặc cURL	Thay thế	Test nhanh từ terminal, tự động hóa smoke test
DBeaver/pgAdmin	Tùy chọn	Đối chiếu dữ liệu trong 4 database
RabbitMQ Management	Tùy chọn	Kiểm tra exchange ecommerce.events, queue order.payment-status.v1 và DLQ
k6	Tùy chọn	Load test và kiểm tra chống overselling
IntelliJ IDEA/VS Code	Tùy chọn	Đọc log, chạy Gradle test hoặc debug service

2.1 Điều kiện trước khi test

- Docker Desktop đang chạy và lệnh docker version trả về cả Client lẫn Server.
- Các cổng 8080, 15432, 15672, 5672, 9090 và 3000 không bị ứng dụng khác chiếm.
- Workspace hiện tại là D:\BaoVe_Microservice.
- File .env đã được tạo từ .env.example và không dùng mật khẩu production cho môi trường demo.

3. Khởi động và kiểm tra môi trường

3.1 Tạo file cấu hình môi trường

```
cd D:\BaoVe_Microservice
Copy-Item .env.example .env
notepad .env
```

> **Lưu ý:** Đặt POSTGRES_PASSWORD và RABBITMQ_PASSWORD khác giá trị mẫu nếu môi trường có thể được truy cập từ máy khác.

3.2 Build và chạy toàn bộ hệ thống

```
docker compose up --build -d
docker compose ps
```

Pass: Các container ở trạng thái Up; postgres và rabbitmq hiển thị healthy sau thời gian khởi động.

3.3 Theo dõi log

```
docker compose logs -f --tail=200 product-service inventory-service order-service payment-service
gateway
```

Lỗi thường gặp: Flyway/schema validation, sai password database, RabbitMQ authentication, cổng bị chiếm hoặc Docker daemon chưa chạy.

3.4 Smoke check

```
curl.exe -i http://localhost:8080/api/products
curl.exe -i http://localhost:15672
```

Pass: GET /api/products trả 200; RabbitMQ UI mở được trang đăng nhập.

3.5 Reset dữ liệu khi cần chạy lại từ đầu

```
docker compose down -v
docker compose up --build -d
```

> **Cảnh báo phá hủy dữ liệu:** docker compose down -v xóa volume PostgreSQL và Grafana của project. Chỉ dùng cho môi trường test khi chắc chắn không cần giữ dữ liệu.

4. Cấu hình Postman

4.1 Import API contract

1. Mở Postman → Import → Files.
2. Chọn D:\BaoVe_Microservice\docs\openapi.yaml.
3. Chọn Generate a collection và xác nhận server là http://localhost:8080.
4. Bổ sung các endpoint GET/POST/DELETE còn thiếu trong OpenAPI theo mục 6-9 của tài liệu này.

> **Giới hạn contract:** File OpenAPI hiện mô tả các endpoint cốt lõi nhưng chưa liệt kê đầy đủ GET product by SKU, GET inventory, GET reservation by order, add/remove order item và GET payment. Danh sách đầy đủ trong tài liệu lấy trực tiếp từ controller hiện tại.

4.2 Tạo Environment

Biến	Giá trị khởi tạo	Cách sinh
baseUrl	http://localhost:8080	Nhập cố định
sku	DEMO-LAPTOP-001	Nhập cố định
userId	11111111-1111-4111-8111-111111111111	Nhập cố định

Biến	Giá trị khởi tạo	Cách sinh
productId		Response tạo product
orderId		Response tạo order
orderItemId		items[0].id của order
reservationId		items[0].inventoryReservationId
paymentId		Response tạo payment
totalAmount		Response tạo order
currency	VND	Response tạo order
orderKey	order-demo-001	Mỗi logical order một key
paymentKey	payment-demo-001	Mỗi logical payment một key
reservationKey	reservation-demo-001	Mỗi logical reservation một key

4.3 Header chung

```
Content-Type: application/json
Accept: application/json
```

Header bắt buộc thêm: Idempotency-Key cho POST reservation, POST order và POST payment. Độ dài 1-128 ký tự.

4.4 Script Postman để lưu ID

```
const j = pm.response.json();
if (j.id) pm.environment.set('productId', j.id);
if (j.sku) pm.environment.set('sku', j.sku);
```

Cho response order: Dùng script dưới đây trong tab Tests của request Create Order.

```
const j = pm.response.json();
pm.environment.set('orderId', j.id);
pm.environment.set('totalAmount', j.totalAmount);
pm.environment.set('currency', j.currency);
if (j.items && j.items.length) {
  pm.environment.set('orderItemId', j.items[0].id);
  pm.environment.set('reservationId', j.items[0].inventoryReservationId);
}
```

Cho response payment: pm.environment.set('paymentId', pm.response.json().id);

5. Dữ liệu mẫu và happy path end-to-end

Trường	Giá trị mẫu	Ghi chú
SKU	DEMO-LAPTOP-001	Được chuẩn hóa thành chữ hoa
Tên sản phẩm	Laptop Demo 14	Snapshot sang OrderItem
Giá	12990000	Số nguyên long; VND
Tồn đầu	10	onHand=10, reserved=0
Số lượng mua	2	Tổng tiền kỳ vọng 25980000
userId	11111111-1111-4111-8111-111111111111	UUID hợp lệ
Payment method	CARD	Enum hợp lệ

5.1 Bước 1 - Tạo sản phẩm

POST {{baseUrl}}/api/products

Mục đích: Tạo product ACTIVE.

Body mẫu:

```
{
  "sku": "DEMO-LAPTOP-001",
  "name": "Laptop Demo 14",
  "description": "Sản phẩm dùng cho kiểm thử API",
  "price": 12990000,
  "currency": "VND"
}
```

Kết quả mong đợi: 201 Created; lưu id vào {{productId}}; status=ACTIVE, version=0.

5.2 Bước 2 - Tạo tồn kho

POST {{baseUrl}}/api/inventory/stock

Mục đích: Tạo inventory item gắn bằng productId thô, không có FK xuyên service.

Body mẫu:

```
{
  "productId": "{{productId}}",
  "sku": "{{sku}}",
  "onHand": 10
}
```

Kết quả mong đợi: 201 Created; onHand=10, reserved=0.

5.3 Bước 3 - Tạo order và giữ hàng

Header: Idempotency-Key: {{orderKey}}

POST {{baseUrl}}/api/orders

Mục đích: Order Service lấy giá/tên/SKU từ Product Service rồi reserve Inventory.

Body mẫu:

```
{
  "userId": "{{userId}}",
  "currency": "VND",
  "items": [
    { "productId": "{{productId}}", "quantity": 2 }
  ]
}
```

Kết quả mong đợi: 201 Created; status=INVENTORY_RESERVED; totalQuantity=2; totalAmount=25980000; item có inventoryReservationId.

5.4 Bước 4 - Kiểm tra reservation

GET {{baseUrl}}/api/inventory/reservation/by-order/{{orderId}}

Mục đích: Đối chiếu reservation do Order Service tạo.

Kết quả mong đợi: 200 OK; một reservation ACTIVE, quantity=2, expiresAt khoảng 15 phút sau createdAt.

GET {{baseUrl}}/api/inventory/{{sku}}

Mục đích: Đối chiếu số lượng giữ hàng.

Kết quả mong đợi: 200 OK; onHand=10, reserved=2, available suy ra là 8.

5.5 Bước 5 - Tạo payment

Header: Idempotency-Key: {{paymentKey}}

POST {{baseUrl}}/api/payments

Mục đích: Tạo payment sau khi Payment Service kiểm tra amount/currency với Order Service.

Body mẫu:

```
{
  "orderId": "{{orderId}}",
  "amount": {{totalAmount}},
  "currency": "{{currency}}",
  "paymentMethod": "CARD"
}
```

Kết quả mong đợi: 201 Created; status=PENDING; lưu id vào {{paymentId}}. Sau khoảng 1-3 giây order chuyển PAYMENT_PROCESSING.

5.6 Bước 6 - Xác nhận payment

POST {{baseUrl}}/api/payments/{{paymentId}}/confirm

Mục đích: Chuyển payment PENDING → SUCCEEDED và phát outbox event.

Body mẫu:

```
{
  "providerReference": "VNPAY-DEMO-0001"
}
```

Kết quả mong đợi: 200 OK; payment.status=SUCCEEDED; providerReference được lưu.

5.7 Bước 7 - Chờ event và xác minh trạng thái cuối

1. Chờ 1-3 giây.
2. GET /api/orders/{{orderId}}: status phải là CONFIRMED.
3. GET /api/inventory/{{sku}}: onHand=8 và reserved=0.
4. GET /api/inventory/reservation/by-order/{{orderId}}: status=CONFIRMED.
5. GET /api/payments/{{paymentId}}: status=SUCCEEDED.

> **Điểm pass quan trọng:** Confirm reservation phải đồng thời giảm onHand và reserved đúng quantity. Với dữ liệu mẫu: (10,2) → (8,0), không phải (10,0).

6. Test Product Service

Method	Path	Kỳ vọng chính
POST	/api/products	201; chuẩn hóa SKU/currency
GET	/api/products	200; danh sách
GET	/api/products/{id}	200 hoặc 404
GET	/api/products/by-sku/{sku}	200; tìm không phân biệt hoa/thường
PUT	/api/products/{id}	200; 409 nếu ARCHIVED

6.1 API đọc và cập nhật

GET {{baseUrl}}/api/products

Mục đích: Liệt kê product.

Kết quả mong đợi: 200 OK; response là JSON array.

GET {{baseUrl}}/api/products/{{productId}}

Mục đích: Lấy product theo UUID.

Kết quả mong đợi: 200 OK; id, sku, name, description, price, currency, status, createdAt, updatedAt, version.

Ca âm nên chạy: UUID không tồn tại → 404 RESOURCE_NOT_FOUND; UUID sai định dạng → 400 từ Spring MVC.

GET {{baseUrl}}/api/products/by-sku/demo-laptop-001

Mục đích: Xác minh chuẩn hóa SKU.

Kết quả mong đợi: 200 OK; response.sku=DEMO-LAPTOP-001.

PUT {{baseUrl}}/api/products/{{productId}}

Mục đích: Cập nhật managed entity; currency và SKU không đổi.

Body mẫu:

```
{
  "name": "Laptop Demo 14 - Updated",
  "description": "Đã cập nhật mô tả",
  "price": 13990000,
  "status": "ACTIVE"
}
```

Kết quả mong đợi: 200 OK; version tăng; id/sku/currency giữ nguyên.

Ca âm nên chạy: Đưa status thành ARCHIVED rồi PUT lần nữa → 409 INVALID_PRODUCT_STATE.

6.2 Ca validation/duplicate

Tình huống	Request thay đổi	Kỳ vọng
Trùng SKU	POST lại cùng DEMO-LAPTOP-001	409 CONCURRENT_OR_DUPLICATE_UPDATE
SKU sai	sku = 'DEMO SKU !'	400 VALIDATION_FAILED
Giá âm	price = -1	400 VALIDATION_FAILED
Currency sai	currency = 'VN'	400 VALIDATION_FAILED

Tình huống	Request thay đổi	Kỳ vọng
Tên rỗng	name = ''	400 VALIDATION_FAILED

7. Test Inventory Service

Method	Path	Kỳ vọng chính
POST	/api/inventory/stock	201; reserved=0
GET	/api/inventory/{sku}	200 hoặc 404
PATCH	/api/inventory/stock/{sku}	Điều chỉnh onHand
POST	/api/inventory/reservation	201; cần Idempotency-Key
PATCH	/api/inventory/reservation/{id}/status	ACTIVE → trạng thái cuối
GET	/api/inventory/reservation/by-order/{orderId}	Danh sách reservation

7.1 API tồn kho và reservation

PATCH {{baseUrl}}/api/inventory/stock/{{sku}}

Mục đích: Nhập thêm hoặc giảm onHand nhưng không được thấp hơn reserved.

Body mẫu:

```
{
  "quantityDelta": 5
}
```

Kết quả mong đợi: 200 OK; onHand tăng 5; version tăng.

Ca âm nên chạy: Gửi quantityDelta âm làm onHand < reserved hoặc < 0 → 409 INVENTORY_CONFLICT.

POST {{baseUrl}}/api/inventory/reservation

Mục đích: Test trực tiếp reserve, độc lập với Order Service.

Body mẫu:

```
{
  "orderId": "22222222-2222-4222-8222-222222222222",
  "sku": "{{sku}}",
  "quantity": 1
}
```

Kết quả mong đợi: Header Idempotency-Key: {{reservationKey}}; 201; status=ACTIVE; reserved tăng 1.

Ca âm nên chạy: Thiếu header hoặc quantity=0 → 400; quantity > available → 409; SKU không tồn tại → 404.

PATCH {{baseUrl}}/api/inventory/reservation/{{reservationId}}/status

Mục đích: Confirm/release/expire reservation.

Body mẫu:

```
{
  "status": "RELEASED"
}
```

Kết quả mong đợi: 200 OK; cùng target gửi lại vẫn 200 và không trừ reserved lần hai.

Ca âm nên chạy: Sau RELEASED gửi CONFIRMED → 409; status=PENDING hoặc null → 400.

7.2 Ma trận tác động tồn kho

Thao tác	Trước	Sau	Ý nghĩa
Reserve q=2	onHand=10, reserved=0	10, 2	available giảm còn 8
Confirm q=2	10, 2	8, 0	đã bán/xuất kho
Release q=2	10, 2	10, 0	trả lại available
Expire q=2	10, 2	10, 0	scheduler hết hạn

7.3 Test expiration

- Cách nhanh: tạo reservation rồi PATCH status=EXPIRED; xác minh reserved giảm đúng quantity.
- Cách kiểm tra scheduler: tạo reservation ACTIVE, không chuyển trạng thái, chờ hơn 15 phút cộng chu kỳ scheduler khoảng 30 giây; GET by-order phải thấy EXPIRED.
- Sau EXPIRED, gửi CONFIRMED hoặc RELEASED phải nhận 409.

8. Test Order Service

8.1 API và state guard

POST {{baseUrl}}/api/orders

Mục đích: Tạo order từ productId + quantity; server tự lấy snapshot giá/tên/SKU và reserve kho.

Body mẫu:

```
{
  "userId": "{{userId}}",
  "currency": "VND",
  "items": [
    { "productId": "{{productId}}", "quantity": 2 }
  ]
}
```

Kết quả mong đợi: 201; INVENTORY_RESERVED; totalQuantity bằng tổng quantity; totalAmount bằng tổng unitPrice × quantity.

Ca âm nên chạy: Hai item cùng productId → 409; currency khác product → 409; product INACTIVE/ARCHIVED → 409; inventory không đủ hoặc unavailable hiện được bọc thành lỗi downstream 503.

GET {{baseUrl}}/api/orders/{{orderId}}

Mục đích: Đọc aggregate đầy đủ.

Kết quả mong đợi: 200; items chứa productId, inventoryReservationId, sku, productName, quantity, unitPrice, lineTotal.

Ca âm nên chạy: Order không tồn tại → 404 RESOURCE_NOT_FOUND.

POST {{baseUrl}}/api/orders/{{orderId}}/items

Mục đích: Thử state guard khi thêm item.

Body mẫu:

```
{
  "productId": "{{productId}}",
  "quantity": 1
}
```

Kết quả mong đợi: Với order tạo thành công (đã INVENTORY_RESERVED), kỳ vọng 409 ORDER_CONFLICT vì chỉ PENDING mới sửa được.

DELETE {{baseUrl}}/api/orders/{{orderId}}/items/{{orderItemId}}

Mục đích: Thử state guard khi xóa item.

Kết quả mong đợi: Với order đã reserve, kỳ vọng 409. Nếu order PENDING thì không được xóa item cuối cùng.

8.2 Test authoritative snapshot

1. Tạo Product giá 12990000.
2. Tạo Order quantity=2 và xác minh unitPrice=12990000, lineTotal=25980000.
3. Update Product lên 13990000.
4. GET Order cũ: unitPrice/lineTotal vẫn giữ snapshot cũ.
5. Tạo Order mới: unitPrice phải dùng giá mới.

8.3 Test idempotency

- Gửi lại đúng payload với cùng Idempotency-Key → 201 và cùng order id; inventory không reserve thêm lần nữa.
- Gửi payload khác với cùng key → 409 ORDER_CONFLICT.
- Đổi thứ tự items nhưng dữ liệu logic giống nhau → request hash chuẩn hóa theo productId; vẫn là cùng request.

9. Test Payment Service

9.1 API và state transition

POST {{baseUrl}}/api/payments

Mục đích: Tạo payment khi order có thể thanh toán và amount/currency khớp tuyệt đối.

Body mẫu:

```
{
  "orderId": "{{orderId}}",
  "amount": {{totalAmount}},
  "currency": "{{currency}}",
  "paymentMethod": "CARD"
}
```

Kết quả mong đợi: 201; PENDING; outbox event payment.status.pending được tạo.

Ca âm nên chạy: Sai amount/currency, order FAILED/CONFIRMED, hoặc order đã có PENDING/SUCCEEDED payment → 409.

GET {{baseUrl}}/api/payments/{{paymentId}}

Mục đích: Lấy payment theo id.

Kết quả mong đợi: 200; trả trạng thái và các thông tin provider/failure/timestamps.

Ca âm nên chạy: ID không tồn tại → 404.

GET {{baseUrl}}/api/payments/by-order/{{orderId}}

Mục đích: Liệt kê payment theo order.

Kết quả mong đợi: 200; JSON array, có thể rỗng.

POST {{baseUrl}}/api/payments/{{paymentId}}/confirm

Mục đích: PENDING → SUCCEEDED.

Body mẫu:

```
{
  "providerReference": "VNPAY-DEMO-0001"
}
```

Kết quả mong đợi: 200; paidAt khác null; event làm order CONFIRMED và inventory reservation CONFIRMED.

Ca âm nên chạy: Confirm lại cùng reference → 200 idempotent; reference khác sau SUCCEEDED → 409; providerReference trùng payment khác → 409.

POST {{baseUrl}}/api/payments/{{paymentId}}/cancel

Mục đích: PENDING → FAILED.

Body mẫu:

```
{
  "reasonCode": "CUSTOMER_CANCELLED",
  "reason": "Khách hàng hủy giao dịch"
}
```

Kết quả mong đợi: 200; failureCode/failureMessage được lưu; event làm order FAILED và release inventory.

Ca âm nên chạy: Hủy payment SUCCEEDED/REFUNDED → 409; reason rỗng → 400.

POST {{baseUrl}}/api/payments/{{paymentId}}/refund

Mục đích: SUCCEEDED → REFUNDED.

Kết quả mong đợi: 200; retry vẫn 200. Refund không tự động nhập lại tồn kho vì hoàn tiền không đồng nghĩa hoàn hàng vật lý.

Ca âm nên chạy: Refund PENDING hoặc FAILED → 409.

9.2 Ma trận state transition

Trạng thái hiện tại	Thao tác	Kết quả
PENDING	confirm	SUCCEEDED
PENDING	cancel	FAILED
SUCCEEDED	refund	REFUNDED
SUCCEEDED	confirm cùng reference	SUCCEEDED, idempotent
FAILED	cancel cùng code/reason	FAILED, idempotent
REFUNDED	refund	REFUNDED, idempotent
Mọi transition còn lại	-	409 PAYMENT_CONFLICT

10. Test lỗi, duplicate request và race condition

10.1 HTTP status cần assert

Status	Ý nghĩa trong hệ thống	Ví dụ
200	Đọc/cập nhật thành công	GET, PUT, PATCH, confirm/cancel/refund
201	Tạo resource thành công	Product, stock, reservation, order, payment
400	Validation hoặc request/header sai	UUID sai, field rỗng, enum sai
404	Không tìm thấy resource	Product/inventory/order/payment không tồn tại
409	Conflict dữ liệu/trạng thái/idempotency/concurrency	Trùng SKU, thiếu stock, transition sai
429/503	Rate limit hoặc downstream unavailable	Gateway quá tải; service phụ thuộc lỗi

> **Problem Details:** Lỗi ứng dụng trả application/problem+json, thường có type, title, status, detail và code. Assert status + code, không chỉ assert chuỗi message tiếng Việt.

10.2 Idempotency test pattern

1. Gửi request lần 1 với key cố định; lưu resource id và trạng thái.
2. Gửi lại cùng key và payload; assert cùng id, không phát sinh bản ghi/stock mutation lần hai.
3. Gửi cùng key nhưng đổi một field nghiệp vụ; assert 409.
4. Gửi payload ban đầu với key mới; assert tạo logical request mới nếu state cho phép.

```
Idempotency-Key: inventory-reserve-test-001
Idempotency-Key: order-create-test-001
Idempotency-Key: payment-create-test-001
```

10.3 Chống overselling

- Seed một SKU với onHand=50, reserved=0.
- Gửi đồng thời ít nhất 100 reservation, mỗi request quantity=1 và key duy nhất.
- Kết quả: tối đa 50 request 201; phần còn lại 409; reserved không vượt onHand.
- GET inventory và SQL phải thỏa 0 <= reserved <= on_hand.

> **Rate limit Gateway:** Nginx giới hạn location inventory khoảng 20 request/giây với burst 80. Để stress lock của Inventory Service ở 500 rps, chạy k6 trong Docker network và target trực tiếp http://inventory-service:8082; nếu target Gateway, nhiều request có thể bị rate-limit trước khi tới service.

```
docker network ls --filter name=baove_microservice_default
docker run --rm --network baove_microservice_default `
  -e BASE_URL=http://inventory-service:8082 `
  -e SKU=FLASH-SALE-SKU -e RATE=500 -e DURATION=30s `
  -v ${PWD}\load-tests:/scripts:ro grafana/k6 run /scripts/inventory-flash-sale.js
```

Thay thế: Nếu tên network khác, lấy tên đúng từ docker network ls. Seed FLASH-SALE-SKU trước khi chạy.

10.4 Race payment confirm/cancel

- Tạo một payment PENDING.
- Gửi confirm và cancel gần như đồng thời từ hai tab/Runner.
- Kỳ vọng chỉ một transition thắng; request còn lại nhận 409 do optimistic locking hoặc invalid state.
- GET payment, order và inventory sau 1-3 giây để xác minh trạng thái cuối nhất quán.

10.5 Test compensation khi order không đủ kho

1. Tạo hai product và inventory; SKU A đủ, SKU B thiếu.
2. Tạo order gồm A trước, B sau để A có thể reserve trước rồi B thất bại.
3. Request order có thể trả 503 do lỗi downstream inventory được wrap.
4. Tra order theo dữ liệu/log/DB: status phải FAILED.
5. GET reservation của order: reservation A phải RELEASED; inventory A reserved trở về giá trị trước request.
6. Nếu compensation tạm lỗi, reservation ACTIVE phải được scheduler expire sau khoảng 15 phút.

11. Xác minh RabbitMQ, PostgreSQL và metrics

11.1 RabbitMQ

Đăng nhập: http://localhost:15672 bằng RABBITMQ_USERNAME/RABBITMQ_PASSWORD trong .env.

- Exchange ecommerce.events tồn tại và là topic exchange.
- Queue order.payment-status.v1 bind với routing key payment.status.#.
- Sau create/confirm/cancel/refund payment, message rate tăng; queue không tích backlog kéo dài khi Order Service healthy.
- Queue order.payment-status.v1.dlq không có message trong happy path.

11.2 PostgreSQL bằng DBeaver/psql

Database	Bảng cần xem	Điểm kiểm tra
product_db	products	SKU/currency uppercase, version, status
inventory_db	inventory_items, inventory_reservations	on_hand, reserved, status, expires_at
order_db	orders, order_items	aggregate totals, snapshot, reservation UUID thô
payment_db	payments, payment_outbox_events	state, provider ref, published_at

```
-- inventory_db
SELECT sku, on_hand, reserved, on_hand - reserved AS available, version
FROM inventory_items WHERE sku = 'DEMO-LAPTOP-001';

SELECT order_id, quantity, status, expires_at
FROM inventory_reservations ORDER BY created_at DESC;

-- order_db
SELECT id, status, total_quantity, total_amount, currency, failure_reason
FROM orders ORDER BY created_at DESC;

-- payment_db
SELECT id, order_id, status, amount, provider_reference, version
FROM payments ORDER BY created_at DESC;

SELECT routing_key, created_at, published_at
FROM payment_outbox_events ORDER BY created_at DESC;
```

> **Ranh giới dữ liệu:** Chỉ query từng database để quan sát. Không join chéo database và không tạo FK từ order/payment sang product/inventory; các ID xuyên service chỉ là UUID thô.

11.3 Metrics

- Mở Prometheus http://localhost:9090 → Status → Targets; các service phải UP.
- Query http_server_requests_seconds_count để xem request count.

- Query rate(http_server_requests_seconds_count[1m]) để quan sát throughput.
- Mở Grafana http://localhost:3000 và cấu hình Prometheus data source nếu chưa có dashboard.

12. Chạy build và test tự động

Chạy từng project: Dùng Gradle Wrapper đi kèm; test profile dùng H2.

```
cd D:\BaoVe_Microservice\product_service; .\gradlew.bat clean test
cd D:\BaoVe_Microservice\inventory_service; .\gradlew.bat clean test
cd D:\BaoVe_Microservice\order_service; .\gradlew.bat clean test
cd D:\BaoVe_Microservice\payment_service; .\gradlew.bat clean test
```

Build đầy đủ: Thay test bằng build để chạy compile, test và đóng gói jar.

```
.\gradlew.bat clean build
```

Nhóm test	Mục tiêu tối thiểu
Unit/domain	Inventory reserve/confirm/release; Order totals; Payment transitions
Mapper	Chuẩn hóa SKU/currency; response field đúng
Repository	Unique constraint, optimistic lock, query lock/expiry
Integration	Flyway + PostgreSQL thực; RabbitMQ outbox/consumer
Contract	Request/response và status giữa 4 service
E2E	Happy path và failure compensation
Load	Không oversell; latency/error rate đạt ngưỡng

13. Checklist nghiệm thu

Đạt	Tiêu chí
	GET /api/products trả 200 qua Gateway.
	Tạo product và inventory thành công; SKU/currency được chuẩn hóa.
	Duplicate SKU và request invalid trả đúng 409/400.
	Tạo order chỉ cần productId + quantity; tổng tiền lấy từ Product Service.
	Order thành công có INVENTORY_RESERVED và mỗi item có inventoryReservationId.
	reserved tăng khi reserve; release/expire giảm reserved; confirm giảm cả onHand và reserved.
	Payment sai amount/currency bị từ chối.

Đạt	Tiêu chí
	Payment PENDING event đưa order sang PAYMENT_PROCESSING.
	Confirm payment đưa payment/order/reservation sang trạng thái thành công cuối.
	Cancel payment đưa order FAILED và release inventory.
	Retry cùng idempotency key không tạo duplicate hoặc mutate stock lần hai.
	Cùng idempotency key nhưng payload khác trả 409.
	Race reservation không làm reserved > onHand.
	RabbitMQ queue không backlog; DLQ trống trong happy path.
	Outbox event có published_at sau khi publish.
	Không có FK/JPA relationship xuyên microservice.
	Build/test của cả bốn project pass.

14. Troubleshooting nhanh

Hiện tượng	Nguyên nhân thường gặp	Cách xử lý
Cannot connect Docker daemon	Docker Desktop chưa chạy	Mở Docker Desktop, chờ Engine ready, chạy docker version
Gateway 502/503	Service chưa sẵn sàng hoặc crash	docker compose ps và logs service tương ứng
Service lỗi Flyway/validate	Schema/volume cũ không tương thích	Đọc log migration; chỉ test mới cân nhắc down -v
RabbitMQ ACCESS_REFUSED	Sai user/password hoặc dùng guest xuyên container	Đồng bộ .env và compose; dùng user ecommerce
Order tạo 503	Product/Inventory unavailable hoặc inventory từ chối	Kiểm tra log order + downstream; kiểm tra tồn SKU
Order chưa CONFIRMED	Event chưa publish/consume	Chờ 1-3 giây; kiểm tra outbox, exchange, queue, DLQ
k6 nhận nhiều 503	Gateway rate limit	Target inventory-service trong Docker network hoặc giảm RATE
Postman gửi literal {{id}}	Environment chưa active/biến rỗng	Chọn đúng environment và kiểm tra Current value
409 khi sửa item order	Order đã INVENTORY_RESERVED	Đây là state guard hiện tại; add/remove chỉ cho PENDING

Phụ lục A - cURL smoke flow

> **Cách dùng:** Các lệnh dưới đây minh họa request độc lập. Với ID động, thay <PRODUCT_ID>, <ORDER_ID>, <PAYMENT_ID> bằng giá trị response thực hoặc dùng Postman để tự lưu biến.

A.1 Tạo product

```
curl.exe -X POST http://localhost:8080/api/products `
  -H "Content-Type: application/json" `
  -d '{"sku":"CURL-DEMO-001","name":"Curl Demo","description":"Smoke
test","price":100000,"currency":"VND"}'
```

A.2 Tạo inventory

```
curl.exe -X POST http://localhost:8080/api/inventory/stock `
  -H "Content-Type: application/json" `
  -d '{"productId":"<PRODUCT_ID>","sku":"CURL-DEMO-001","onHand":5}'
```

A.3 Tạo order

```
curl.exe -X POST http://localhost:8080/api/orders `
  -H "Content-Type: application/json" `
  -H "Idempotency-Key: curl-order-001" `
  -d '{"userId":"11111111-1111-4111-8111-111111111111","currency":"VND","items":
[{"productId":"<PRODUCT_ID>","quantity":1}]}'
```

A.4 Tạo payment

```
curl.exe -X POST http://localhost:8080/api/payments `
  -H "Content-Type: application/json" `
  -H "Idempotency-Key: curl-payment-001" `
  -d '{"orderId":"<ORDER_ID>","amount":100000,"currency":"VND","paymentMethod":"CARD"}'
```

A.5 Confirm payment

```
curl.exe -X POST http://localhost:8080/api/payments/<PAYMENT_ID>/confirm `
  -H "Content-Type: application/json" `
  -d '{"providerReference":"CURL-PROVIDER-001"}'
```

Phụ lục B - Assertions Postman gợi ý

```
pm.test('Status đúng', () => pm.response.to.have.status(201));

const j = pm.response.json();
pm.test('Có id UUID', () => {
  pm.expect(j.id).to.match(/^[0-9a-f-]{36}$/i);
});

pm.test('Không oversell', () => {
  if (j.onHand !== undefined && j.reserved !== undefined) {
    pm.expect(j.reserved).to.be.at.least(0);
    pm.expect(j.reserved).to.be.at.most(j.onHand);
  }
});

pm.test('ProblemDetail hợp lệ khi lỗi', () => {
  if (pm.response.code >= 400) {
    pm.expect(j).to.have.property('status');
    pm.expect(j).to.have.property('code');
    pm.expect(j).to.have.property('detail');
  }
});
```

Kết luận

> **Tiêu chí hoàn tất:** Một vòng test được coi là đạt khi happy path kết thúc với Payment=SUCCEEDED, Order=CONFIRMED, Reservation=CONFIRMED, Inventory onHand giảm đúng quantity và reserved=0; đồng thời các retry/lỗi không tạo duplicate hay làm sai tồn kho.